

The Softmax Function.

The step that turns raw scores into attention weights. We define it, prove it is a valid probability distribution, derive its Jacobian by hand, show the elegant gradient it produces with cross-entropy, and cover the numerical-stability trick every implementation uses.

THE FUNCTION

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad \frac{\partial \text{softmax}(z)_i}{\partial z_j} = A_i(\delta_{ij} - A_j).$$

Worked example. We reuse the four scaled attention scores from Topic 1 and compute the full 4×4 Jacobian, every entry by hand.

Topic 3 of 5 in this module · companion to the M0.1 course page · v1.0

Contents

1	Definition and what it is for	2
2	It is a valid probability distribution	2
3	The Jacobian — derived by hand	3
4	Worked example — the full 4×4 Jacobian	3
5	Softmax + cross-entropy: the gradient collapses	4
6	Numerical stability: subtract the max	5
7	Summary	5
A	Reproducible (NumPy)	5

1 Definition and what it is for

Softmax maps a vector of n real “scores” (also called *logits*) to a vector of n probabilities:

$$\text{softmax}(z)_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}, \quad z \in \mathbb{R}^n. \quad (3)$$

In attention it is applied to each *row* of the scaled score matrix $QK^\top/\sqrt{d_k}$, converting raw similarities into the weights A that blend the values. Two design requirements make it the natural choice:

- **Order-preserving and positive.** $e^{z_i} > 0$ always, and e^x is strictly increasing, so the largest score gets the largest weight — it is a *soft* arg max.
- **Differentiable.** Unlike a hard arg max (which has zero gradient almost everywhere), softmax is smooth, so the model can learn *which scores to raise*.

2 It is a valid probability distribution

Two facts establish that the output lives on the probability simplex.

(i) Non-negativity. Each $e^{z_i} > 0$ and the denominator is a sum of positive terms, so $\text{softmax}(z)_i > 0$ for all i .

(ii) **Sums to one.**

$$\sum_{i=1}^n \text{softmax}(z)_i = \sum_{i=1}^n \frac{e^{z_i}}{\sum_j e^{z_j}} = \frac{\sum_i e^{z_i}}{\sum_j e^{z_j}} = 1.$$

Hence each attention row is a genuine distribution over keys, and (Topic 1) the output is a convex combination of value vectors.

Shift-invariance. For any constant c , $\text{softmax}(z+c\mathbf{1}) = \text{softmax}(z)$, because $e^{z_i+c} = e^c e^{z_i}$ and the e^c cancels top and bottom. Softmax depends only on score *differences*, not absolute levels. This is the property the stability trick in Section 5 exploits, and the reason a causal mask uses $-\infty$ (Topic 5): $e^{-\infty} = 0$ removes a key entirely.

3 The Jacobian — derived by hand

We need $\partial A_i / \partial z_j$ where $A_i = \text{softmax}(z)_i$. Write $A_i = e^{z_i} / D$ with $D = \sum_k e^{z_k}$, and note $\partial D / \partial z_j = e^{z_j}$. Use the quotient rule, splitting into two cases.

Case $i = j$:

$$\frac{\partial A_i}{\partial z_i} = \frac{e^{z_i} D - e^{z_i} e^{z_i}}{D^2} = \frac{e^{z_i}}{D} \cdot \frac{D - e^{z_i}}{D} = A_i (1 - A_i).$$

Case $i \neq j$: the numerator e^{z_i} does not depend on z_j , so

$$\frac{\partial A_i}{\partial z_j} = \frac{0 \cdot D - e^{z_i} e^{z_j}}{D^2} = -\frac{e^{z_i}}{D} \cdot \frac{e^{z_j}}{D} = -A_i A_j.$$

Both cases combine into the compact Kronecker-delta form:

$$\frac{\partial A_i}{\partial z_j} = A_i(\delta_{ij} - A_j), \quad \delta_{ij} = \begin{cases} 1 & i = j \\ 0 & i \neq j. \end{cases} \quad (4)$$

In matrix form the Jacobian is $J = \text{diag}(A) - AA^\top$. It is symmetric, and every *row* sums to zero (a consequence of shift-invariance): $\sum_j J_{ij} = A_i(1 - \sum_j A_j) = A_i(1 - 1) = 0$.

Equation (4) says: pushing logit z_j up steals probability mass from every other class in proportion to how much they currently hold ($-A_i A_j$), while feeding the diagonal term $A_i(1 - A_i)$. The factor $A_i(1 - A_i)$ is largest when $A_i \approx \frac{1}{2}$ and vanishes when $A_i \rightarrow 0$ or $A_i \rightarrow 1$ — which is exactly why a saturated softmax stops learning (the gradient motivation for the $\sqrt{d_k}$ scaling in Topic 2).

4 Worked example — the full 4×4 Jacobian

Take the four scaled scores from Topic 1, $z = [0.5658, 0.0751, 0.8949, -0.1674]$, whose softmax is

$$A = [0.2872, 0.1758, 0.3991, 0.1379].$$

Apply Eq. (4). Diagonal entries $A_i(1 - A_i)$:

$$J_{11} = 0.2872(1 - 0.2872) = 0.2047, \quad J_{22} = 0.1758(0.8242) = 0.1449,$$

$$J_{33} = 0.3991(0.6009) = 0.2398, \quad J_{44} = 0.1379(0.8621) = 0.1189.$$

Off-diagonal entries $-A_i A_j$, e.g. $J_{13} = -A_1 A_3 = -(0.2872)(0.3991) = -0.1146$. The complete matrix:

$$J = \text{diag}(A) - AA^\top = \begin{bmatrix} 0.2047 & -0.0505 & -0.1146 & -0.0396 \\ -0.0505 & 0.1449 & -0.0702 & -0.0243 \\ -0.1146 & -0.0702 & 0.2398 & -0.0551 \\ -0.0396 & -0.0243 & -0.0551 & 0.1189 \end{bmatrix}.$$

Each row sums to 0 (check row 1: $0.2047 - 0.0505 - 0.1146 - 0.0396 = 0.0000$). A finite-difference check confirms entry J_{13} : nudging z_3 by 10^{-6} changes A_1 by -1.1460×10^{-7} , giving numeric $\partial A_1 / \partial z_3 = -0.11460$ — matching the analytic -0.1146 .

5 Softmax + cross-entropy: the gradient collapses

Softmax is almost always paired with the cross-entropy loss, and the combination produces one of the cleanest gradients in deep learning. With a one-hot target y (true class t), the loss is

$$L = -\sum_i y_i \log A_i = -\log A_t.$$

Differentiating through the softmax with Eq. (4) and simplifying:

$$\frac{\partial L}{\partial z_j} = -\sum_i \frac{y_i}{A_i} \frac{\partial A_i}{\partial z_j} = -\sum_i \frac{y_i}{A_i} A_i (\delta_{ij} - A_j) = -\sum_i y_i (\delta_{ij} - A_j) = A_j - y_j.$$

$$\boxed{\frac{\partial L}{\partial z} = A - y}$$

(5)

The gradient is simply “prediction minus target.” For our example, with target class $t = 3$ (one-hot $y = [0, 0, 1, 0]$):

$$\frac{\partial L}{\partial z} = A - y = [0.2872, 0.1758, -0.6009, 0.1379].$$

The correct class gets a negative push (raise its logit), every wrong class gets a positive push (lower its logit), and the magnitudes are exactly the probability errors.

Eq. (5) is why softmax+cross-entropy trains so stably: the messy Jacobian $\text{diag}(A) - AA^\top$ cancels against the $1/A_i$ from the log, leaving a bounded, interpretable signal $A - y$. No exploding terms, no division by small numbers — the gradient is always in $[-1, 1]$ per component.

6 Numerical stability: subtract the max

Computing e^{z_i} directly overflows for large z_i (e.g. $e^{1000} = \infty$ in floating point). Shift-invariance (Section 2) lets us subtract the row maximum $m = \max_j z_j$ first, with *no change* to the result:

$$\text{softmax}(z)_i = \frac{e^{z_i - m}}{\sum_j e^{z_j - m}}.$$

Now the largest exponent is $e^0 = 1$, so nothing overflows, and underflow of tiny terms only rounds them to 0 — harmless. Every production implementation does this:

```
def softmax(z):
    z = z - z.max()          # shift-invariant; prevents overflow
    e = np.exp(z)
    return e / e.sum()
```

Forgetting the max-subtraction is fine on toy inputs but produces **nan** on real logits (which routinely exceed several hundred after the matmul). The shift is free — always include it.

7 Summary

1. Softmax (Eq. 3) maps logits to a positive vector that sums to 1 — a soft, differentiable arg max.
2. It is shift-invariant: it depends only on score differences (hence $-\infty$ masking and the stability trick).
3. Its Jacobian is $J = \text{diag}(A) - AA^\top$, i.e. $A_i(\delta_{ij} - A_j)$ (Eq. 4); rows sum to zero.
4. With cross-entropy the gradient collapses to $A - y$ (Eq. 5) — bounded and interpretable.
5. The diagonal factor $A_i(1 - A_i)$ vanishes when softmax saturates, the gradient reason behind $\sqrt{d_k}$ scaling (Topic 2).

A Reproducible (NumPy)

```
import numpy as np
z = np.array([0.5658, 0.0751, 0.8949, -0.1674])
A = np.exp(z - z.max()); A /= A.sum()          # [0.2872 0.1758 0.3991 0.1379]
J = np.diag(A) - np.outer(A, A)                # full 4x4 Jacobian
print(J.sum(axis=1))                           # rows ~ 0
y = np.array([0, 0, 1.0, 0])                  # target = class 3
print(A - y)                                   # dL/dz = [0.2872 0.1758 -0.6009 0.1379]
```